

GitHub setup and course workflow

14.33 Research and Communication in Economics · Fall 2026

Git tracks changes to files on your computer. GitHub stores a remote copy of the repository and supports collaboration, issues, and links to evidence. These instructions use Git in a terminal; GitHub Desktop is not required.

1. Create the GitHub repository

1. Create a free account at github.com.
2. Create a repository named `fa26-1433-YOUR-KERBEROS`. Replace `YOUR-KERBEROS` with your MIT Kerberos username.
3. Use the repository visibility specified in class. Do not commit confidential data, passwords, API keys, or other secrets.
4. In the repository, select **Settings** → **Collaborators**, then add the instructor and TA using the GitHub usernames provided in class.

This access is required by the [course syllabus](#).

2. Install Git and authentication support

Windows

Install [Git for Windows](#). Keep Git Credential Manager selected during installation.

macOS

If you use [Homebrew](#), run:

```
brew install git
brew install --cask git-credential-manager
```

Git Credential Manager opens a browser when GitHub authentication is required. Follow the browser prompts. See [GitHub's authentication instructions](#) if authentication fails.

Identify your commits

Run these commands once. Replace the example values with your name and the email address associated with GitHub.

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

3. Clone the repository

On the GitHub repository page, select **Code** → **HTTPS**, then copy the URL. In a terminal, run:

```
git clone REPOSITORY-HTTPS-URL
cd fa26-1433-YOUR-KERBEROS
```

Cloning creates a local project directory and connects it to the GitHub repository.

4. Save work with Git

Use this cycle whenever you reach a meaningful checkpoint:

```
git pull --ff-only
git status
git add path/to/file
git diff --staged
git commit -m "Describe the completed change"
git push
```

- `git pull --ff-only` downloads new remote commits without creating an unintended merge commit.
- `git status` shows changed, staged, and untracked files.
- `git add path/to/file` stages a specific file for the next commit.
- `git diff --staged` shows exactly what the next commit will contain.
- `git commit` records the staged changes locally.
- `git push` sends local commits to GitHub.

Use short commit messages that describe completed work, such as `Add baseline summary statistics`. Commit source code, documentation, and small derived outputs needed to understand the project. Do not commit large raw datasets unless the course staff approves them.

5. Use GitHub issues for the course

At least 24 hours before each one-on-one meeting:

1. Open a new issue in your project repository.
2. Title it `Meeting preparation – YYYY-MM-DD`.
3. Complete the template in the `Individual meeting preparation` section of the syllabus. Aim for approximately 250 words, excluding links and tables.
4. Link to supporting evidence in the repository. Report findings and decisions rather than time spent.

Use separate issues for consequential unresolved decisions or incomplete tasks. You may test alternatives on short-lived branches and link those branches from the relevant issue. After each meeting, update the agenda with agreed next steps and record decisions in the linked issues.

6. Correct mistakes without deleting history

If a change has not been committed, edit the file again. To remove a file from the staging area without discarding its changes, run:

```
git restore --staged path/to/file
```

If an incorrect commit is already on GitHub, preserve the shared history. Either make a correcting commit or create a reversal commit:

```
git revert COMMIT-ID
git push
```

Ask the instructor or TA before rewriting published history with commands such as `git reset --hard` or a force push.